

PROJECT DOCUMENTATION

Machine Learning Web Application

Credit Card Fraud Detection

An End-to-End Streamlit Machine Learning Application

Live Application	https://creditcard-fraud-detection-yfqq.onrender.com
Source Dataset	Kaggle -- Credit Card Fraud Detection (mlg-ulb)
Hosting Platform	Render.com (Web Service)
Core Stack	Python, Streamlit, scikit-learn, imbalanced-learn, Plotly
Deployment Status	Live -- verified July 13, 2026
Author	Lokii

1. Project Overview

This document describes the design, features, and deployment of a Streamlit-based machine learning application built around the Kaggle **Credit Card Fraud Detection** dataset. The application takes a user from raw transaction data through exploratory analysis, model training with imbalance-aware evaluation, and live fraud prediction on individual transactions -- all inside a single interactive web interface.

A live instance of the application is deployed on Render.com and reachable at <https://creditcard-fraud-detection-yfqq.onrender.com>.

1.1 Key Capabilities

- Flexible data loading -- use a bundled synthetic sample instantly, or upload the real Kaggle **creditcard.csv** at runtime.
- Interactive data exploration -- shape, dtypes, class balance, descriptive statistics, missing-value checks.
- Visual EDA dashboard -- class distribution, amount and time distributions split by class, per-feature box plots, correlation heatmap.
- Configurable model training -- Logistic Regression, Random Forest, or Gradient Boosting, each combinable with three imbalance-handling strategies.
- Imbalance-aware evaluation -- accuracy, precision, recall, F1, ROC-AUC, PR-AUC, confusion matrix, ROC curve, and precision-recall curve.
- Live fraud prediction -- select or randomize a transaction and view the trained model's classification and fraud probability in real time.
- Production deployment -- configured and verified on Render.com, with Streamlit Community Cloud supported as an alternative.

2. Dataset

The application is built around the [Credit Card Fraud Detection dataset](#) published by the Machine Learning Group (ULB) on Kaggle. It contains 284,807 anonymized European card transactions recorded over two days in September 2013, of which 492 (approximately 0.172%) are fraudulent. Because the underlying features are commercially sensitive, 28 of them have been reduced to principal components via PCA and are shared only as V1 through V28.

2.1 Schema

Column	Description
Time	Seconds elapsed between this transaction and the first transaction in the dataset
V1 -- V28	Principal components from a PCA transformation of the original (withheld) features

Column	Description
Amount	Transaction amount, in the original currency unit
Class	Target label: 0 = legitimate transaction, 1 = fraudulent transaction

2.2 Bundled Sample Data

The real dataset is not redistributed in this project, since it is a large (~150 MB) file distributed under Kaggle's own terms. Instead, the app ships with a small synthetic sample (`data/sample_creditcard.csv`) generated to match the real schema and general shape of the data, so the application is fully functional immediately after setup. In the deployed instance captured for this document, the bundled sample contained **2,000 transactions**, of which **8 (0.400%)** were labeled fraudulent -- a proportion deliberately kept close to the real dataset's own imbalance.

For genuine results, users download **creditcard.csv** from Kaggle and either place it in the `data/` folder or upload it directly through the app's sidebar file uploader, with no restart or redeploy required.

Note: metrics and screenshots throughout this document reflect the bundled synthetic sample and are intended to demonstrate the application's functionality, not to represent production-grade fraud-detection performance.

3. System Architecture

The application follows a simple, modular structure that separates data handling, modeling, and visualization from the Streamlit page logic itself.

```

creditcard-fraud-detection/
├── app.py                Streamlit app entry point (all pages)
├── src/
│   ├── data_loader.py    load / validate / clean / scale / split
│   ├── model.py          training, resampling, metrics
│   └── visualization.py  Plotly chart builders
├── data/
│   ├── generate_sample_data.py regenerates the synthetic sample
│   └── sample_creditcard.csv bundled synthetic demo dataset
├── models/              trained models saved here (.pkl)
├── .streamlit/config.toml theme + server settings
├── requirements.txt
├── Procfile / render.yaml deployment configuration
└── README.md

```

3.1 Technology Stack

Layer	Technology	Purpose
Interface	Streamlit	Interactive multi-page web UI
Modeling	scikit-learn	Logistic Regression, Random Forest, Gradient Boosting

Layer	Technology	Purpose
Imbalance handling	imbalanced-learn	SMOTE oversampling, random undersampling
Visualization	Plotly	Interactive charts (distributions, ROC, PR, confusion matrix)
Data handling	pandas / NumPy	Loading, cleaning, feature scaling
Persistence	joblib	Saving and loading trained models
Hosting	Render.com	Production web service deployment

4. Application Walkthrough

The following screenshots were captured directly from the live deployment at <https://creditcard-fraud-detection-yfqq.onrender.com> and illustrate the application's main pages in the order a typical user would move through them.

4.1 Home Page

The landing page introduces the project, links to the source dataset on Kaggle, and surfaces four key metrics as soon as data is loaded: total transactions, fraud count, fraud rate, and the active data source. An inline notice reminds the user when they are viewing the bundled synthetic sample rather than real transaction data, followed by a quick preview of the raw table.

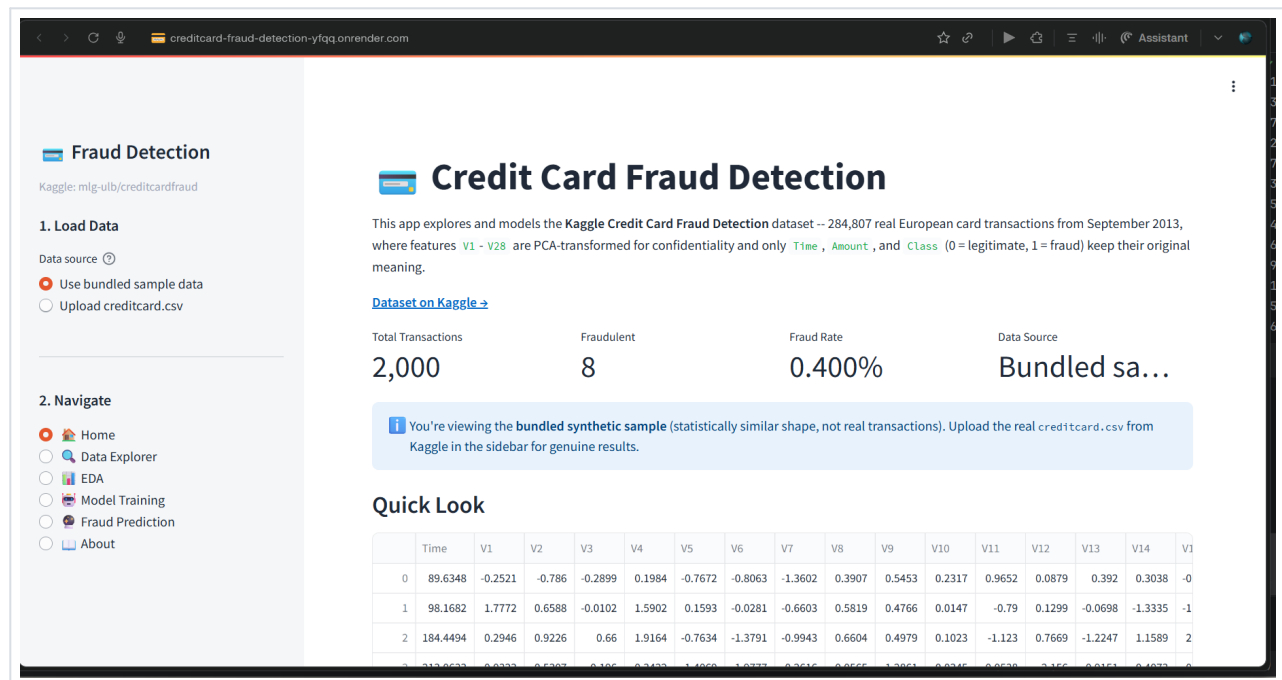


Figure 1 -- Home page showing dataset summary metrics (2,000 transactions, 8 fraud cases, 0.400% fraud rate) and a live data preview.

4.2 Exploratory Data Analysis

The EDA page renders the class distribution as a bar chart, the transaction amount distribution split by class on a log scale with an overlaid box plot, and a hurdle-free view of transaction timing by hour of day. Further down, users can inspect any of the 28 PCA features by class and build a custom correlation heatmap.

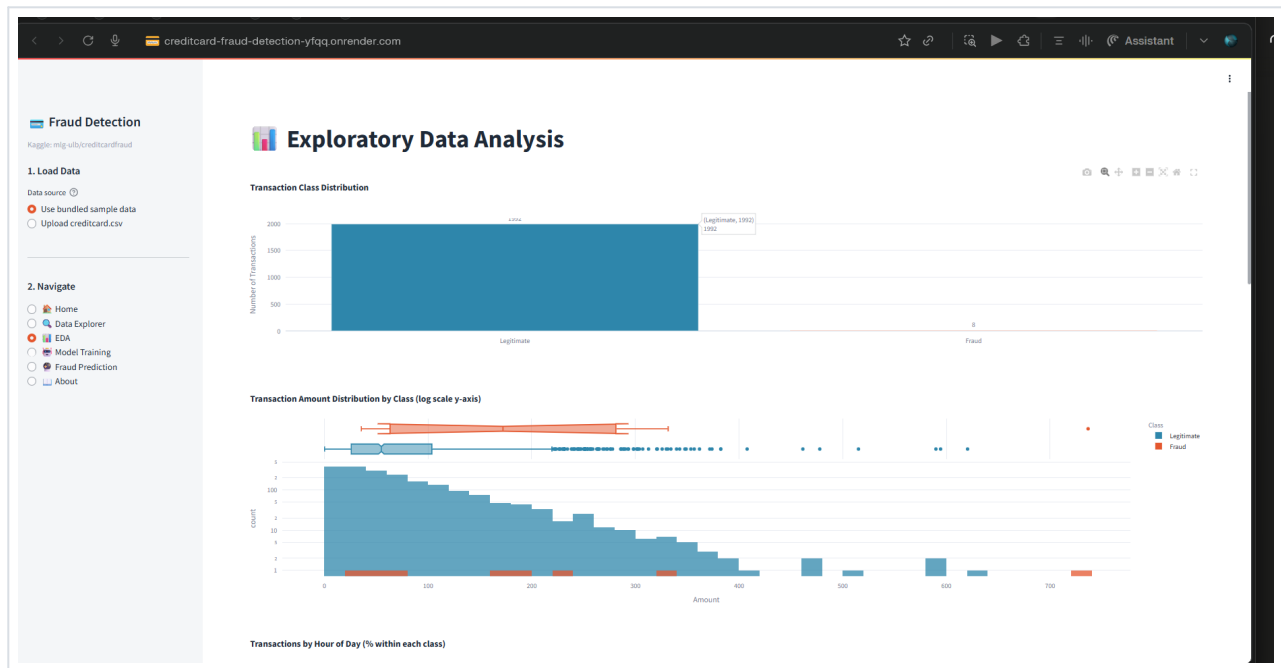


Figure 2 -- EDA dashboard: class distribution and amount distribution by class, immediately surfacing how rare and how differently-sized fraudulent transactions are.

4.3 Model Training & Evaluation

Users choose an algorithm, an imbalance-handling strategy, and a test split size, then train with a single click. Results are reported with metrics chosen specifically for imbalanced classification, alongside a confusion matrix and ROC curve. In the run captured below, a Logistic Regression model trained with no resampling correctly classified all 400 test transactions (398 legitimate, 2 fraud), yielding perfect scores across every metric on this particular sample.

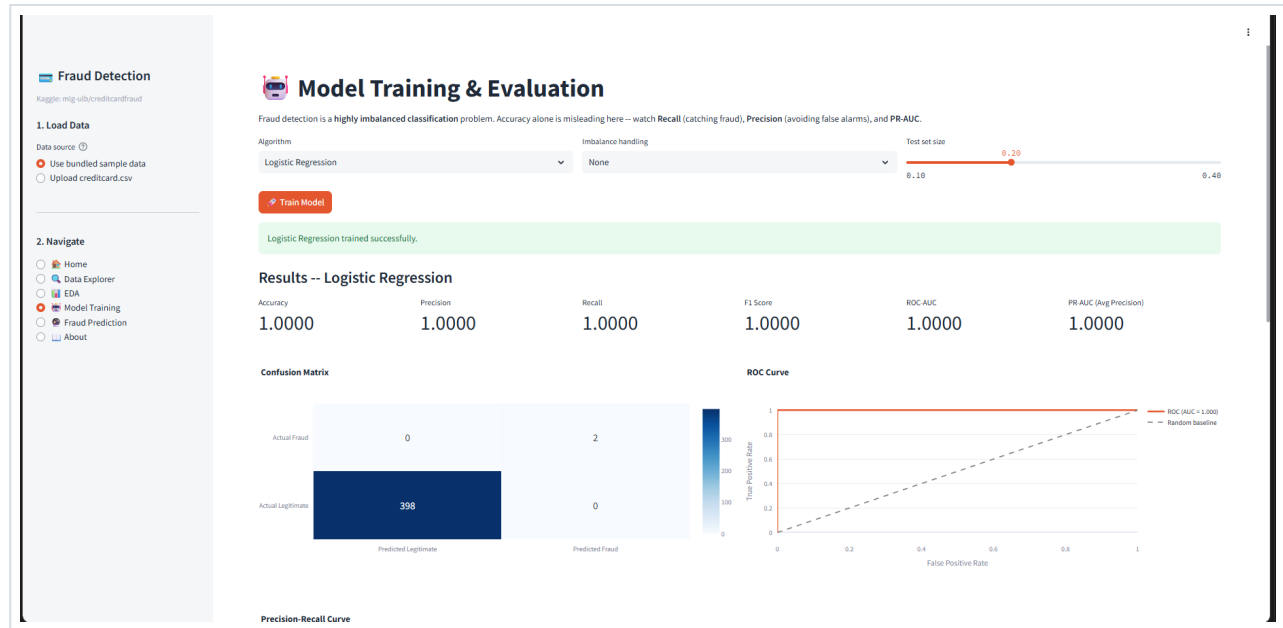


Figure 3 -- Model Training & Evaluation page: metrics panel, confusion matrix (0 missed fraud cases), and ROC curve for a Logistic Regression run.

4.4 Fraud Prediction

Once a model is trained, the Fraud Prediction page lets a user step through the loaded dataset by row index or pull a random transaction, then view the model's predicted class, fraud probability, and the true label side by side for quick sanity-checking.

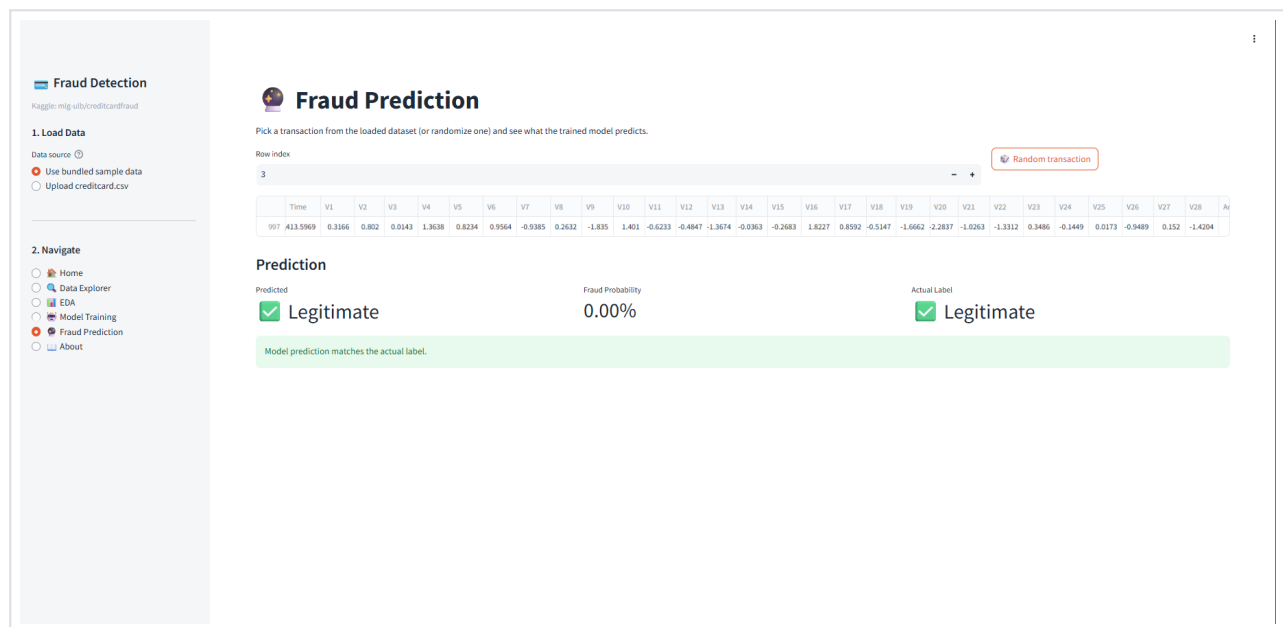


Figure 4 -- Fraud Prediction page: a sampled transaction correctly classified as Legitimate with a 0.00% predicted fraud probability, matching its actual label.

5. Deployment

The application is deployed as a Render.com Web Service, built directly from the project's requirements.txt and started with the command in the project's Procfile:

```
streamlit run app.py --server.port=$PORT --server.address=0.0.0.0
```

Render injects the \$PORT environment variable at runtime, which Streamlit binds to directly -- no additional configuration is required beyond the build and start commands above (also captured declaratively in render.yaml for one-click Blueprint deployment).

Live Deployment Log

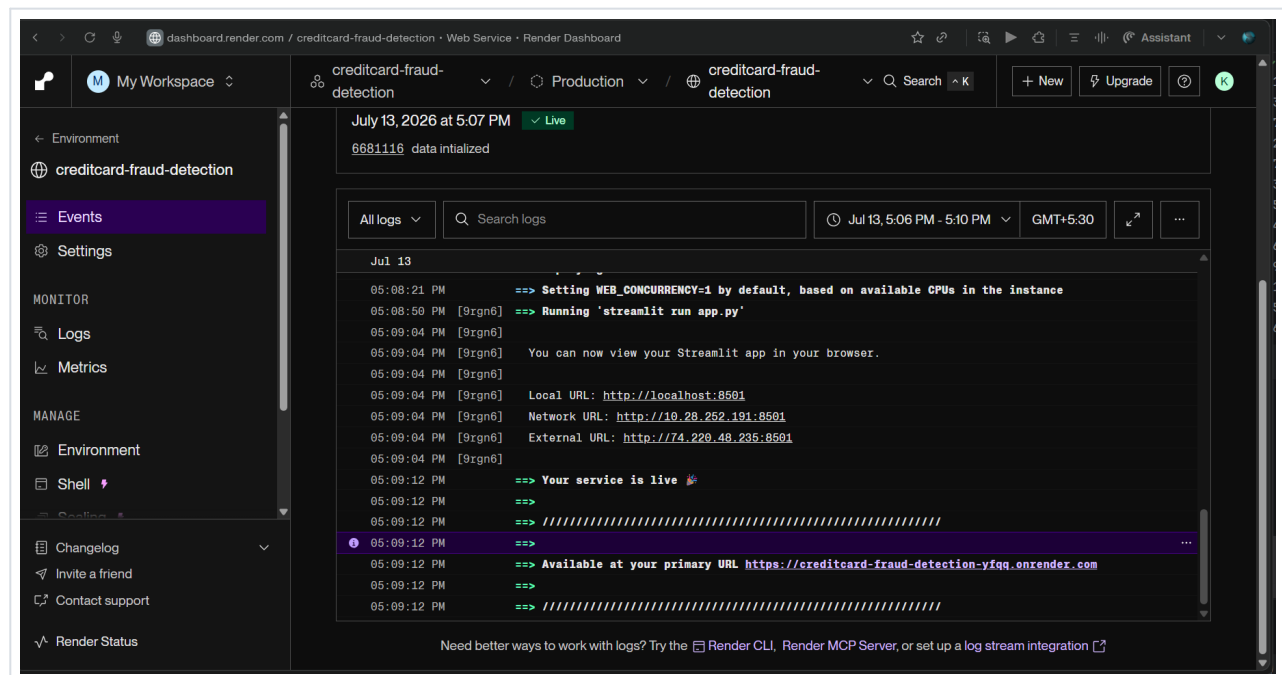


Figure 5 -- Render deployment log confirming a successful build and launch on July 13, 2026 at 5:09 PM, with the service published at <https://creditcard-fraud-detection-yfqq.onrender.com>.

5.1 Deployment Checklist

- Build command: `pip install -r requirements.txt`
- Start command: `streamlit run app.py --server.port=$PORT --server.address=0.0.0.0`
- Python version pinned via runtime.txt (3.11.9)
- Theme and server behavior configured in .streamlit/config.toml
- Real dataset intentionally excluded from version control (see .gitignore); supplied at runtime via the sidebar uploader

Streamlit Community Cloud is supported as a free alternative deployment target using the same requirements.txt, requiring only that the repository be pushed to GitHub and pointed to from share.streamlit.io.

6. Modeling Methodology

Fraud detection is a highly imbalanced classification problem -- in the real dataset, fraud accounts for roughly 0.17% of all transactions. A model that predicts "legitimate" for every transaction would exceed 99.8% accuracy while catching zero fraud, which is why this application is built to surface metrics that remain meaningful under imbalance.

6.1 Evaluation Metrics

Metric	What it captures
Precision	Of all transactions flagged as fraud, what share truly were fraud
Recall	Of all actual fraud cases, what share the model successfully caught
F1 Score	Harmonic mean of precision and recall
ROC-AUC	Overall ability to rank fraud above legitimate transactions
PR-AUC	Precision/recall trade-off area -- more informative than ROC-AUC under heavy class imbalance

6.2 Imbalance-Handling Strategies

- **None** -- train on the natural class distribution as a baseline.
- **Class weighting** -- penalize misclassifying the minority (fraud) class more heavily during training.
- **SMOTE oversampling** -- synthetically generate new minority-class examples to balance the training set.
- **Random undersampling** -- randomly reduce the majority (legitimate) class to match the minority class size.

Critically, resampling is applied to the training split only -- the test split always retains the real class proportions, so reported metrics reflect performance under realistic conditions rather than an artificially balanced evaluation set.

6.3 Algorithms

Three scikit-learn classifiers are available from the Model Training page: Logistic Regression, Random Forest, and Gradient Boosting. The underlying registry (src/model.py) is written so additional scikit-learn-compatible estimators can be added with a single new entry.

7. Local Setup

Clone or unzip the project, then from the project root:

```
python -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate
```

```
pip install -r requirements.txt
```

```
streamlit run app.py
```

The app opens at <http://localhost:8501>. Use the sidebar to switch between the bundled sample and an uploaded copy of the real Kaggle dataset.

8. Future Enhancements

- Automated dataset retrieval via the Kaggle API, removing the manual download/upload step.
- Additional algorithms (e.g. XGBoost, LightGBM) added to the model registry.
- Persistent model storage and versioning across deployments.
- Dockerfile and CI pipeline for containerized, automatically-tested deployments.
- Batch prediction support for scoring an uploaded file of many transactions at once.

9. References

Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). *Calibrating Probability with Undersampling for Unbalanced Classification*. IEEE Symposium on Computational Intelligence and Data Mining (CIDM).

Dataset: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

Live application: <https://creditcard-fraud-detection-yfqq.onrender.com>